

Лабораторна робота №3

Тема: Рядки у мові Python.

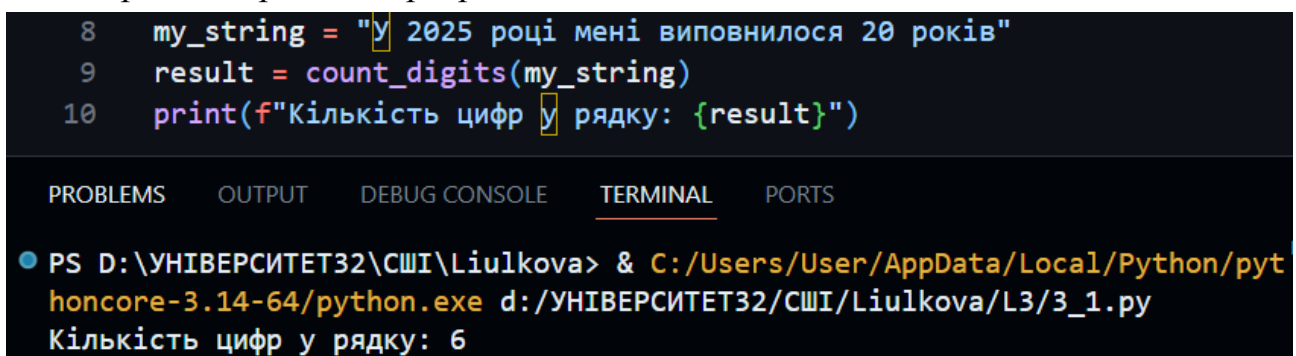
Мета: Набуття практичних навичок створення програм обробки рядків у мові Python.

Лабораторні завдання

Завдання 1. Скласти програму, яка містить функцію, що визначає кількість цифр у заданому рядку.

Опис коду розробленої програми

Програма використовує функцію `count_digits`, яка приймає рядок як аргумент. У циклі функція перевіряє кожен символ рядка за допомогою вбудованого методу `isdigit()`. Якщо символ є цифрою, лічильник збільшується на одиницю. Результат виконання програми та підрахунку цифр продемонстровано на рис. 1.1.

Приклади роботи програми:

```
8 my_string = "У 2025 році мені виповнилося 20 років"
9 result = count_digits(my_string)
10 print(f"Кількість цифр у рядку: {result}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/python-core-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_1.py
Кількість цифр у рядку: 6
```

Рис. 1.1 Результат роботи програми для підрахунку цифр у заданому рядку

Завдання 2. Скласти програму, яка містить функцію, що визначає кількість входжень у заданий рядок деякий підрядок, не використовуючи метод `count()`.

Опис коду розробленої програми

Функція `count_substring` визначає кількість входжень підрядка, проходячи по головному рядку циклом. На кожній ітерації програма створює зріз (slice) довжиною, що дорівнює шуканому підрядку, і порівнює їх. Якщо вони збігаються, лічильник збільшується. Виведення результату пошуку підрядка показано на рис. 2.1.

Приклад роботи програми:

```

14 text = "abcaabababcc"
15 sub = "abc"
16 result = count_substring(text, sub)
17 print(f"Підрядок '{sub}' зустрічається {result} раз(и)")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_2.py
Підрядок 'abc' зустрічається 2 раз(и)

```

Рис. 2.1 Демонстрація пошуку входжень підрядка у рядок за допомогою зрізів

Завдання 3. Дано речення, в якому слова розділені одним або декількома пропусками. Скласти програму, яка визначає і виводить на екран список (без повторень) всіх його слів.

Опис коду розробленої програми

Програма розбиває вхідне речення на окремі слова за допомогою методу `split()`, який автоматично ігнорує множинні пропуски. Для того, щоб залишити лише унікальні слова, список конвертується у словник (де ключі завжди унікальні) і назад у список, що дозволяє уникнути повторень. Сформований список слів виводиться на екран (рис. 3.1).

Приклади роботи програми:

```

7 my_sentence = " у цьому реченні існують існують кілька кілька однакових слів слів"
8 result = get_unique_words(my_sentence)
9 print("Список унікальних слів:")
10 print(result)

```

```

(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_3.py
Список унікальних слів:
['у', 'цьому', 'реченні', 'існують', 'кілька', 'однакових', 'слів']

```

Рис. 3.1 Виведення списку унікальних слів з речення з множинними пропусками

Завдання 4. Дано слово. Скласти програму, яка перевіряє, чи є воно паліндромом (паліндром читається однаково в обох напрямках, наприклад “шалаш”). Алгоритм роботи програми повинен бути заснований на порівнянні першого символу слова з останнім, другого символу – з передостаннім і т. д.

Опис коду розробленої програми

Програма використовує цикл `for`, який проходить лише до середини слова. На кожній ітерації i -тий символ з початку порівнюється з i -тим символом з кінця (використовується індекс `length - 1 - i`). Якщо хоча б одна пара не

збігається, функція повертає False. Результат перевірки тестового слова наведено на рис. 4.1.

Приклади роботи програми:

```

9   test_word = "шалаш"
10  if is_palindrome(test_word):
11      print(f"Слово '{test_word}' є паліндромом.")
12  else:
13      print(f"Слово '{test_word}' не є паліндромом.")
14
● /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/LiulkoVA/L3/3_4.py
Слово 'шалаш' є паліндромом.

```

Рис. 4.1 Результат роботи програми: перевірка слова на паліндром шляхом посимвольного порівняння

Завдання 5. Дано рядок і множина символів. Скласти програму, що визначає, які з символів заданої множини зустрічаються в ньому частіше.

Опис коду розробленої програми

Програма ітерується по заданій множині символів і рахує їхню кількість у рядку за допомогою методу count(), зберігаючи результати у словник. Далі алгоритм знаходить максимальне значення серед підрахованих кількостей і формує список тих символів, які відповідають цьому максимуму. Успішне знаходження найчастішого символу зображено на рис. 5.1.

Приклади роботи програми:

```

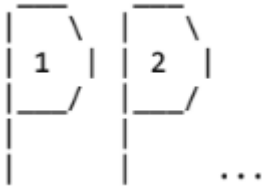
15  my_string = "програмування на python це круто!"
16  my_set = {'a', 'o', 'n', 'p'}
17  result = most_frequent_from_set(my_string, my_set)
18  print(f"Найчастіші символи з множини: {result}")

● /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/LiulkoVA/L3/3_5.py
Найчастіші символи з множини: ['p', 'a']

```

Рис. 5.1 Демонстрація пошуку найчастіших символів із заданої множини у введеному тексті

Завдання 6. Скласти програму, яка по введеному числу **n** від 1 до 9 виводить на екран **n** літер **P** з відповідними номерами. Зображення однієї літери має розмір 6 x 6 символів, між двома сусідніми літерами також є порожній (з пропусків) стовпець. У середині кожної літери повинен бути записаний її номер – число від 1 до **n**.



Опис коду розробленої програми

Оскільки консоль виводить текст по рядках зверху вниз, програма генерує зображення не по одній букві, а по одному горизонтальному "зрізу" для всіх літер одразу. Створюється 6 текстових змінних (по одній на кожен рядок матриці 6x6). Для рядка з цифрами використовується генератор списків, який підставляє номер *i*. Сформований ASCII-арт відображається на екрані рис. 6.1.

Приклади роботи програми:

```

13      print(row1)
14      print(row2)
15      print(row3)
16      print(row4)
17      print(row5)
18      print(row6)
19
20      print_p_letters(6)

```

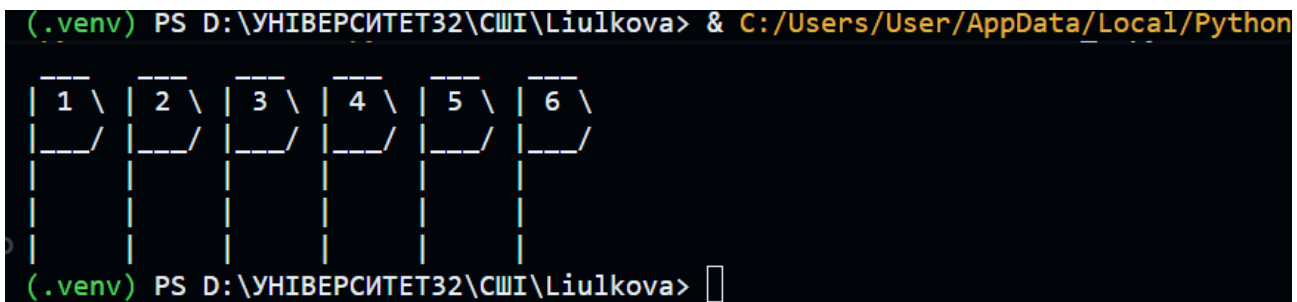


Рис. 6.1 Згенеровані літери Р із вписаними номерами від 1 до n

Завдання 7. Скласти програму, яка визначає, чи є введене слово ідентифікатором за наступним правилом: слово починається з літери англійського алфавіту (малої або великої) або знака підкреслення і не містить інших символів, крім букв англійського алфавіту (в будь-якому регістрі), цифр і знака підкреслення.

Опис коду розробленої програми

Для того, щоб уникнути пропускання кириличних символів вбудованими методами Python, програма використовує явно задані рядки з дозволеними

символами (eng_letters та цифри). Спочатку перевіряється нульовий символ рядка — чи входить він до множини valid_first (англійські літери + підкреслення). Далі в циклі перевіряються всі інші символи на входження до множини valid_all (додаються цифри). Якщо хоч один символ не збігається — перевірка відхиляється. Приклад роботи алгоритму з кирилицею та цифрами показано на рис. 7.1.

Приклади роботи програми:

```
18 test_id_1 = "_myVar123"
19 test_id_2 = "2ndVariable!"
20 test_id_3 = "AnyWord"
21 test_id_4 = "And3ndVariable_"
22
23 print(f"'{test_id_1}' є ідентифікатором: {is_valid_identifier(test_id_1)}")
24 print(f"'{test_id_2}' є ідентифікатором: {is_valid_identifier(test_id_2)}")
25 print(f"'{test_id_3}' є ідентифікатором: {is_valid_identifier(test_id_3)}")
26 print(f"'{test_id_4}' є ідентифікатором: {is_valid_identifier(test_id_4)}")
```

```
o /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_7.py
'_myVar123' є ідентифікатором: True
'2ndVariable!' є ідентифікатором: False
'AnyWord' є ідентифікатором: True
'And3ndVariable_' є ідентифікатором: False
```

Рис. 7.1 Результати валідації введених слів на відповідність правилам іменування ідентифікаторів

Завдання 8. Кіт Леопольд відправив мишам опис, як знайти скарб. Опис складається з рядків виду: North 7, де перше слово – одне з North, South, East, West, а друге число – кількість кроків, яку потрібно пройти в цьому напрямку. Напишіть програму, яка за описом шляху до скарбу визначає координати скарбу, вважаючи, що початок координат знаходиться на початку шляху, вісь Ох спрямована на схід, вісь Oy – на північ. Програма отримує на вхід послідовність рядків зазначеного виду, а введення завершується рядком зі словом Treasure!. Програма має вивести два цілих числа в один рядок з пропуском між ними – координати скарбу.

Наприклад, нехай вхідними даними є:

```
East 4
North 7
East 2
South 2
Treasure!
```

Тоді результат роботи програми повинен бути таким:

Опис коду розробленої програми

Програма працює у нескінченному циклі while, зчитуючи команди до появи ключового слова Treasure!. Рядок розбивається на напрямок і кількість кроків. Залежно від напрямку, змінюються значення координат x (East/West) або y (North/South). Підсумковий результат обчислень відображено на рис. 8.1.

Приклади роботи програми:

```

• /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_8.py
Введіть шлях (наприклад, 'North 7'). Для завершення введіть 'Treasure!':
North 7
East 4
South 2
East 2
Treasure!
Координати скарбу: 6 5

```

Рис. 8.1 Розрахунок підсумкових координат після введення послідовності кроків

Завдання 9. Написати програму з функцією, яка отримує рядок із 16 цифр і маскує їх, залишаючи видимими лише останні чотири (формат: **** * 1234).

Опис коду розробленої програми

Програма перевіряє довжину та вміст вхідного рядка. Якщо рядок коректний, функція використовує зріз [-4:], щоб отримати останні чотири цифри. Після цього за допомогою f-рядка (форматування) формується новий рядок із зірочками та збереженим закінченням номера. Результат роботи функції форматування показано на рис. 9.1.

Приклади роботи програми:

```

9   card = "1111222233334444"
10  masked = mask_card_number(card)
11  print(f"Оригінал: {card}")
12  print(f"Замасковано: {masked}")

• (.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python
/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_9.py
Оригінал: 1111222233334444
Замасковано: **** * 4444

```

Рис. 9.1 Результат роботи програми з маскування 16-значного числового рядка

Завдання 10. Написати програму з функцією, яка приймає речення і повертає нове, де кожне слово перевернуте (реверсоване), але порядок слів залишається незмінним.

Опис коду розробленої програми

Програма розділяє речення на масив окремих слів за допомогою методу `split()`. Далі в циклі (генераторі списків) кожне слово перевертається за допомогою зрізу кроком `-1` (`[::-1]`). Наостанок метод `join()` об'єднує змінені слова назад у єдиний текст. Демонстрація перевертання слів наведена на рис. 10.1.

Приклади роботи програми:

```
7 text = "привіт світ це тестс номер 10"
8 result = reverse_words_in_sentence(text)
9 print(f"Вхідне речення: {text}")
10 print(f"Результат: {result}")
```

```
• /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/LiulkovalL3/3_10.py
Вхідне речення: привіт світ це тестс номер 10
Результат: тівирп тівс ец стсет ремон 01
```

Рис. 10.1. Виведення речення, у якому символи кожного слова розміщені у зворотному порядку

Завдання 11. Скласти програму, яка обчислює відсоткове співвідношення голосних та приголосних літер у введеному тексті.

Опис коду розробленої програми

Функція переводить текст у нижній регістр та ітерується по кожному символу. Метод `isalpha()` відсіює пропуски та розділові знаки. Якщо літера присутня у завчасно створеному рядку голосних, збільшується їх лічильник, інакше — лічильник приголосних. Відсоток вираховується діленням на загальну кількість літер. Результат обчислень відсотків показано на рис. 11.1.

Приклади роботи програми:

```
25 my_text = "Програмувати на Python це круто!"
26 v_pct, c_pct = calculate_letters_percentage(my_text)
27 print(f"Голосних: {v_pct:.1f}%")
28 print(f"Приголосних: {c_pct:.1f}%")
```

```
• /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/LiulkovalL3/3_11.py
Голосних: 40.7%
Приголосних: 59.3%
```

Рис. 11.1 Результат роботи програми: розрахунок відсоткового співвідношення літер

Завдання 12. Написати програму з функцією, яка приймає текст і перелік заборонених слів та повертає новий рядок, де кожне таке слово замінене на [ЦЕНЗУРА] незалежно від регістру.

Опис коду розробленої програми

Програма використовує бібліотеку регулярних виразів `re`. Для кожного слова зі списку заборонених створюється патерн пошуку, де прапорець `re.IGNORECASE` ігнорує регістр, а теги `\b` вказують на межі слова (щоб не замінити випадково частину іншого слова). Метод `sub()` виконує саму заміну знайдених збігів на рядок [ЦЕНЗУРА]. Роботу фільтру цензури наведено на рис. 12.1.

Приклади роботи програми:

```
14 original_text = "Цей Секретний документ містить секретний код та інші дані, які не треба знати."
15 banned = ["секретний", "код"]
16 censored = censor_text(original_text, banned)
17
18 print(f"Оригінал: {original_text}")
19 print(f"Цензура: {censored}")
```

● `/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_12.py`
 Оригінал: Цей Секретний документ містить секретний код та інші дані, які не треба знати.
 Цензура: Цей [ЦЕНЗУРА] документ містить [ЦЕНЗУРА] [ЦЕНЗУРА] та інші дані, які не треба знати.

Рис. 12.1 Результат роботи програми: заміна заборонених слів у тексті на спеціальний маркер

Завдання 13. Написати програму з двома функціями: перша приймає рядок і повертає його стиснену версію за методом Run-Length Encoding, де після кожного символу вказується кількість його безперервних повторень, а друга – виконує зворотне перетворення, відновлюючи початковий текст із такого закодованого рядка.

Наприклад, перша функція має перетворити «aaaabbbccccdbbbb» на «a3b2c4d1b3», а друга – навпаки.

Опис коду розробленої програми

Перша функція `rlc_encode` порівнює поточний символ із наступним, накопичуючи лічильник повторень. Друга функція `rlc_decode` проходить по рядку з кроком 2, зчитуючи символ і цифру-кількість для відновлення тексту. Приклад стиснення та відновлення даних продемонстровано на рис. 13.1.

Приклади роботи програми:


```

● /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_13.py
Оригінал: aaabbcccccdbbb
Стиснений: a3b2c4d1b3
Відновлений: aaabbcccccdbbb
○ (.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova>

```

Рис. 13.1 Результат роботи функцій кодування та декодування рядка за методом RLE

Завдання 14. Виконати завдання за своїм варіантом:

скласти програму, яка за вибором користувача у заданому рядку змінює:

- всі маленькі літери (англійські, українські і російські), що містяться в ньому, на великі;
- всі великі - на маленькі;
- змінює регістр літер;

Опис коду розробленої програми

Програма використовує вбудовані методи Python upper(), lower() та swapcase(), які коректно працюють з англійською та кириличною (українською/російською) абетками. Користувач обирає режим через консольне меню, після чого рядок трансформується. Приклад зміни регістру наведено на рис. 14.1.

Приклади роботи програми:

```

● /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_14_13.py
Оберіть дію:
1 - Все у великі літери
2 - Все у маленькі літери
3 - Змінити регістр (Swap case)
Ваш вибір: 1
Результат: ПРОГРАМУВАТИ НА PYTHON ЦЕ КРУТО! ТЕСТ РЕГІСТРУ

```

```

● /pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_14_13.py
Оберіть дію:
1 - Все у великі літери
2 - Все у маленькі літери
3 - Змінити регістр (Swap case)
Ваш вибір: 2
Результат: програмувати на python це круто! тест регістру

```

```

• (.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python
/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_14_13.py
Оберіть дію:
1 - Все у великі літери
2 - Все у маленькі літери
3 - Змінити регістр (Swap case)
Ваш вибір: 3
Результат: ПРОГРАМУВАТИ НА PYTHON ЦЕ КРУТО! ТЕСТ РЕГІСТРУ

/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L3/3_14_13.py
• Оберіть дію:
1 - Все у великі літери
2 - Все у маленькі літери
3 - Змінити регістр (Swap case)
Ваш вибір: 0
Результат: Неправильний вибір
    
```

Рис. 14.1 Демонстрація зміни регістру тексту для різних алфавітів за вибором користувача

Висновки. Під час виконання лабораторної роботи було опановано методи роботи з рядковими даними в Python. Реалізовано алгоритми для пошуку підрядків, аналізу символного складу (підрахунок цифр, голосних/приголосних), валідації ідентифікаторів та маскування даних. Також було вивчено методи стиснення інформації (RLE) та принципи обробки тексту (Unicode) при зміні регістрів.